تحليل وتصميم الخوارزميات

المحاضرة الثالثة: الصيغ التقاربية وBig-O Notation

في المحاضرة السابقة، تعلمنا كيفية تحليل الخوارزميات من خلال عد العمليات الأساسية، وتعرفنا على حالات الأداء (الأفضل، الأسوأ، والمتوسط). فهمنا إن في الخوارزميات يهمنا حقاً ليس العدد الدقيق للعمليات، بل كيف ينمو وقت التنفيذ مع زيادة حجم المدخلات.(n)

هنا يأتي دور **الصيغ التقاربية .(Asymptotic Notations)** إنها أدوات رياضية تسمح لنا بالتركيز على "الصورة الكبيرة" لأداء الخوارزمية وتجاهل التفاصيل غير المهمة.

ما هي الصيغ التقاربية؟ ولماذا نستخدمها؟

تخيل أن لديك خوارزميتين لحل نفس المشكلة:

- **الخوارزمية :A** تحتاج إلى n^2عملية.
- **الخوارزمية :B** تحتاج إلى $100n + 50$عملية.

عندما يكون حجم المدخلات صغيراً مثلاً $n=10$:

- الخوارزمية A ستستغرق $10^2 = 100$عملية.
- الخوارزمية B ستستغرق $100 * 10 + 50 = 1050$عملية. هنا تبدو الخوارزمية A أفضل.

لكن ماذا يحدث عندما يصبح حجم المدخلات كبيراً جداً (مثلاً $n=1,000,000$)؟

- الخوارزمية A ستستغرق $(1,000,000)^2 = 1,000,000,000,000$عملية (تريليون).
- الخوارزمية B ستستغرق $100 * 1,000,000 + 50 = 100,000,050$عملية (حوالي 100 مليون).

من الواضح أن الخوارزمية B هي الأفضل بكثير للمدخلات الكبيرة. الصيغ التقاربية تساعدنا على فهم هذا "السلوك طويل الأمد" للخوارزمية.

أشهر الصيغ التقاربية

هناك ثلاث صيغ رئيسية نستخدمها لوصف تعقيد الخوارزميات:

1. **صيغة الحد الأعلى :(Big-O Notation - O)** تصف **الحالة الأسوأ .(Worst-Case)** وهي الأكثر شيوعاً واستخداماً.
2. **صيغة الحد الأدنى :(Omega Notation - Ω)** تصف **الحالة الأفضل.(Best-Case)**

3. **صيغة الحد المحكم :(Theta Notation - Θ)** تصف الحالة التي يكون فيها أداء الخوارزمية محصوراً بين الحدين الأعلى والأدنى (الحالة المتوسطة غالباً).

سنركز اليوم على الصيغة الأهم والأكثر استخداماً.**Big-O Notation** :

Big-O Notation

Big-O هي طريقة لوصف **الحد الأعلى** لنمو دالة التعقيد. بعبارة أخرى، هي تخبرنا بأن أداء الخوارزمية **لن يكون أسوأ من** معدل نمو معين.

قواعد :Big-O

عندما نحول دالة تعقيد (مثل $4n^2 + 100n + 500$) إلى صيغة Big-O ، نتبع قاعدتين بسيطتين:

1. **تجاهل المعاملات الثابتة :(Drop Constants)** نتجاهل الأرقام المضروبة في n. لأننا نهتم بمعدل النمو وليس بالرقم الدقيق.
2. **الاحتفاظ بالحد الأعلى فقط :(Keep the Highest Order Term)** نهتم فقط بالحد الذي ينمو بشكل أسرع كلما كبرت n.

مثال تطبيقي :لنفترض أن دالة التعقيد لخوارزمية ما هي $f(n) = 4n^2 + 100n + 500$:

1. **الخطوة الأولى :**نحدد الحد الأعلى (الأسرع نمواً). لدينا n^2 و n و (500ثابت . من الواضح أن n^2 هو الأسرع نمواً.
2. **الخطوة الثانية :**نتجاهل الحدود الأقل أهمية، فتصبح الدالة $4n^2$.
3. **الخطوة الثالثة :**نتجاهل المعامل الثابت (الرقم 4).

النتيجة :نقول إن تعقيد هذه الخوارزمية هو (**$O(n^2)$** تُنطق."Big-O of n squared")

أشهر تعقيدات Big-O من الأسرع إلى الأبطأ

1. **- O(1) تعقيد ثابت:(Constant Time)**

1. **الوصف :**الأداء ثابت ولا يعتمد على حجم المدخلات. هذا هو أفضل تعقيد ممكن.
2. **مثال :**الوصول إلى عنصر في مصفوفة باستخدام الفهرس.(e.g., my_array[5])

2. **- O(log n) تعقيد لوغاريتمي:(Logarithmic Time)**

0. **الوصف :**الأداء ينمو ببطء شديد. كلما تضاعف حجم المدخلات، يزداد الوقت بمقدار ثابت.
1. **مثال :**خوارزمية البحث الثنائي (Binary Search) التي درسناها سابقاً.

3. **- O(n) تعقيد خطي:(Linear Time)**

0. **الوصف :**ينمو وقت التنفيذ بشكل طردي مع حجم المدخلات.

1. **مثال :**البحث عن عنصر في مصفوفة غير مرتبة، أو جمع عناصر مصفوفة.

4. **- O(n log n)تعقيد خطي-لوغاريتمي:(Linearithmic Time)**

0. **الوصف :**أسرع قليلاً من التعقيد الخطي. شائع جداً في خوارزميات الترتيب الفعالة.

1. **مثال :**خوارزميات الترتيب المتقدمة مثل الفرز المدمج (Merge Sort) والفرز السريع.(Quick Sort)

5. **- $O(n^2)$تعقيد تربيعي:(Quadratic Time)**

0. **الوصف :**ينمو الأداء بشكل كبير مع زيادة حجم المدخلات. يحدث عادة عند وجود حلقة تكرارية متداخلة.(nested loop)

1. **مثال :**خوارزميات الترتيب البسيطة مثل الفرز الفقاعي.(Bubble Sort)

6. **- $O(2^n)$تعقيد أسي:(Exponential Time)**

0. **الوصف :**الأداء ينمو بشكل هائل ويصبح غير عملي بسرعة كبيرة حتى مع المدخلات الصغيرة.

1. **مثال :**بعض الحلول غير الفعالة لمسائل معقدة مثل إيجاد متتالية فيبوناتشي باستخدام الاستدعاء الذاتي (Recursion) دون تحسين.

أمثلة تطبيقية على Big-O Notation

دعونا الآن نطبق القواعد التي تعلمناها على بعض الأمثلة العملية لنرى كيف يمكننا تحديد تعقيد Big-Oلخوارزمية ما.

المثال الأول: تعقيد طباعة كل الأرقام الزوجية الممكنة في مصفوفة

السؤال :ما هو تعقيد Big-O لخوارزمية تقوم بطباعة كل الأرقام الزوجية الممكنة من العناصر في مصفوفة حجمها n؟

الشرح والتحليل:

للقيام بهذه المهمة، نحتاج إلى المرور على كل عنصر في المصفوفة، ولكل عنصر، نحتاج إلى المرور مرة أخرى على كل عناصر المصفوفة لإقرانه بها. هذا يتطلب بشكل طبيعي استخدام حلقتين تكراريتين متداخلتين.(nested loops)

لنرَ كيف يبدو الكود بلغة البايثون:

Python

```
def print_all_pairs(my_list):
```

```
n = len(my_list)
# الحلقة الخارجية تمر على كل عنصر
for i in range(n):
# الحلقة الداخلية تمر على كل عنصر مرة أخرى
    for j in range(n):
        print(f"({my_list[i]}, {my_list[j]})")
```

حساب التعقيد:

1. الحلقة الخارجية ((for i in range(n)) ستدور n من المرات.
2. مقابل كل دورة من الحلقة الخارجية، ستدور الحلقة الداخلية n ((for j in range(n)) من المرات.
3. إذن، العدد الإجمالي لعمليات الطباعة (print) التي سيتم تنفيذها هو nمن الحلقة الخارجية) مضروباً في) من الحلقة الداخلية.
4. العدد الإجمالي للعمليات.n^2 = n * n =

بما أن عدد العمليات ينمو بمقدار مربع حجم المدخلات(n^2) ، فإن تعقيد هذه الخوارزمية هو $O(n^2)$تعقيد تربيعي.

خلاصة :عندما ترى في كود ما حلقة تكرارية داخل حلقة أخرى، وكلاهما يعتمد على حجم المدخلات n، فهذا مؤشر قوي على أن التعقيد هو .$O(n^2)$

المثال الثاني: ترتيب التعقيدات من الأفضل إلى الأسوأ

السؤال :رتب التعقيدات التالية من الأفضل (الأسرع) إلى الأسوأ (الأبطأ) O(n), O(n log : n), $O(n^2)$, O(log n), O(1).

الشرح والترتيب:

الترتيب يعتمد على مدى سرعة نمو وقت التنفيذ كلما زاد حجم المدخلات " .nالأفضل" يعني الأقل نمواً، و"الأسوأ" يعني الأكثر نمواً.

1. **- O(1) تعقيد ثابت:(Constant)**

هو الأفضل على الإطلاق. الأداء لا يتغير مهما زاد حجم المدخلات. إنه سريع جداً.

2. **- O(log n) تعقيد لوغاريتمي:(Logarithmic)**

ممتاز. الوقت يزداد ببطء شديد. حتى مع ملايين العناصر، عدد العمليات يبقى صغيراً جداً.

3. **- O(n) تعقيد خطي:(Linear)**

جيد جداً. الوقت يزداد بشكل معقول ومتوقع مع زيادة حجم المدخلات.

4. **- O(n log n) تعقيد خطي-لوغاريتمي:(Linearithmic)**

جيد. أبطأ قليلاً من الخطي، ولكنه لا يزال فعالاً جداً ويستخدم في أشهر خوارزميات الترتيب.

5. **- $O(n^2)$ تعقيد تربيعي:(Quadratic)**

مقبول للمدخلات الصغيرة فقط. يصبح بطيئاً جداً وغير عملي مع زيادة حجم المدخلات بشكل ملحوظ.

الترتيب النهائي (من الأفضل إلى الأسوأ):

$O(1) \rightarrow O(\log n) \rightarrow O(n) \rightarrow O(n \log n) \rightarrow O(n^2)$

المثال الثالث: تبسيط دالة تعقيد إلى صيغةBig-O

السؤال :إذا كانت لدينا خوارزمية تستغرق $5n^3 + 2n^2 + 1000$عملية، فما هو تعقيد Big-O الخاص بها؟ اشرح خطواتك.

الشرح والخطوات:

الهدف هنا هو تبسيط التعبير الرياضي للتركيز على معدل النمو الأساسي للخوارزمية عند قيم nالكبيرة جداً. نطبق القاعدتين اللتين تعلمناهما:

الدالة الأصلية$f(n) = 5n^3 + 2n^2 + 1000$:

1. الخطوة الأولى: تحديد الحد الأعلى:(Highest Order Term)

1. ننظر إلى جميع حدود الدالة.$5n^3, 2n^2, 1000$:
2. : أي حد ينمو بشكل أسرع كلما أصبحت nكبيرة جداً؟
3. الحد n^3ينمو أسرع بكثير من n^2، وكلاهما ينمو أسرع من الثابت 1000.
4. لذلك، الحد الأعلى والأكثر تأثيراً هو $5n^3$.

2. الخطوة الثانية: تجاهل الحدود الأقل أهمية:

0. بما أن $5n^3$هو الحد المهيمن، يمكننا تجاهل بقية الحدود ($2n^2$) و (1000لأن تأثيرها يصبح ضئيلاً جداً مقارنة بالحد n^3عندما تكون nكبيرة.

1. يتبقى لدينا.$5n^3$:

3. الخطوة الثالثة: تجاهل المعاملات الثابتة:(Drop Constants)

0. نتجاهل الرقم الثابت المضروب في الحد الأعلى، وهو الرقم 5.

1. يتبقى لدينا: n^3.

النتيجة النهائية:

بعد تطبيق قواعد التبسيط، نجد أن تعقيد Big-O لهذه الخوارزمية هو ($O(n^3)$.هذا يعني أن وقت تنفيذ الخوارزمية ينمو بمعدل تكعيبي نسبةً لحجم المدخلات، وهو يعتبر تعقيداً بطيئاً وغير مرغوب فيه للمدخلات الكبيرة.

خلاصة Big-O Notation **:**هي الأداة الأساسية التي تمكننا من التحدث عن كفاءة الخوارزميات بطريقة موحدة ومفهومة. فهم هذه التعقيدات المختلفة يساعدنا على اختيار الخوارزمية المناسبة للمشكلة التي نحاول حلها، خاصة عندما نتعامل مع كميات كبيرة من البيانات